BookFlix

Taking reading to the next level

Team Members

Documentation
Clever Raimundo
Daniel Borges
Jonathan Magalhães
Matheus Costa

Design & Prototyping
Geovanna Lessa
Jezreel Anderson
Vitor Lima de Carvalho

Development
Daniel Borges
Geovanna Lessa
Marcela Donata

About

Mobile application

Reading plans

Gamified concept

BookFlix is a mobile application that encourages the habit of reading through a gamified reward system.

Users create a reading plan based on suggested books and set a deadline for completion, earning points as they achieve their goals.

Problem

66% of Brazilian young people do not read texts longer than 10 pages

Programme for International Student Assessment (PISA) – 2018

Objective

This project aligns with the United Nations Sustainable Development Goal 4: Quality Education:

Promote lifelong learning opportunities for all

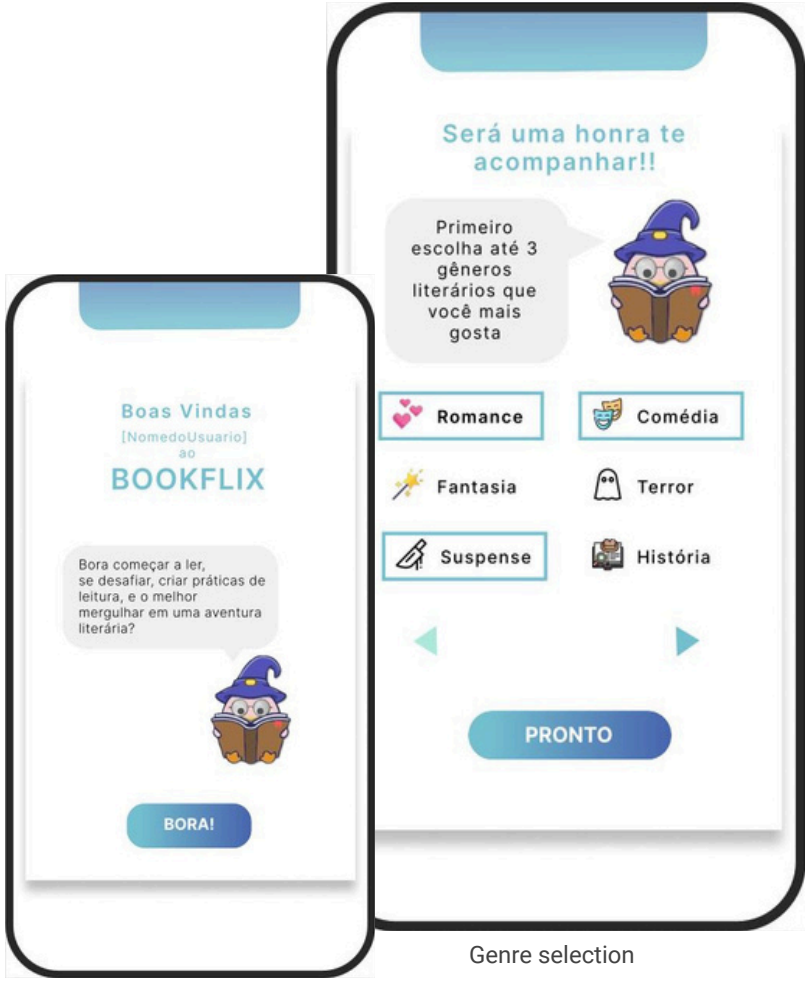
> *Reading habits are strongly linked to mental health. Reading involves imagination, mental processing, anticipation, and continuous learning (at the very least, new vocabulary acquisition), functioning as a form of mental exercise for the human brain.*

AugustoBuchweitz

The app

Screen Prototypes

BookFlix provides users with reading plans, suggesting books based on their preferred literary genres.



Genre selection

Welcome screen

The app

Screen Prototypes

Users must complete books within a specified timeframe to earn points. Points are awarded as users follow and complete their reading plans.

Deadline display

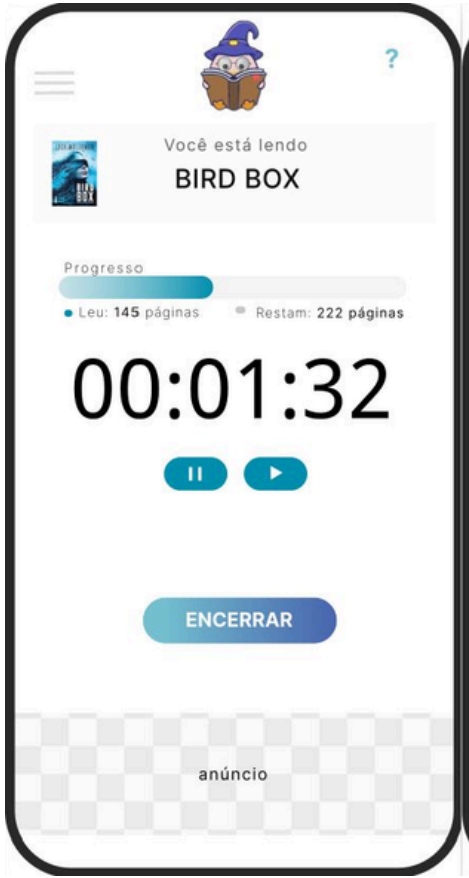
Reading plan selection

The app

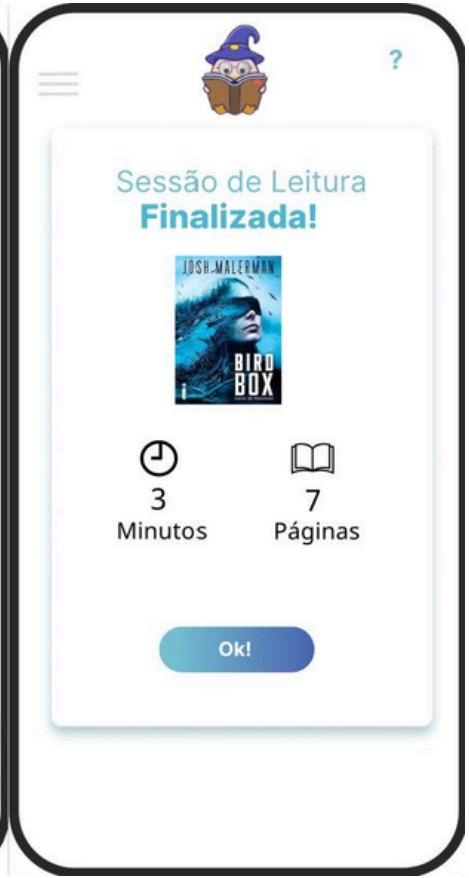
Screen Prototypes

To track progress, the app includes a timer (stopwatch).
At the end of each session, the user records the last page read.

Future plans include implementing a built-in eBook reader with automatic tracking.



Reading session control



End of session

The app

Screen Prototypes

Gamification concepts are applied to the reward system. Users earn points for achievements and level up at specific milestones.

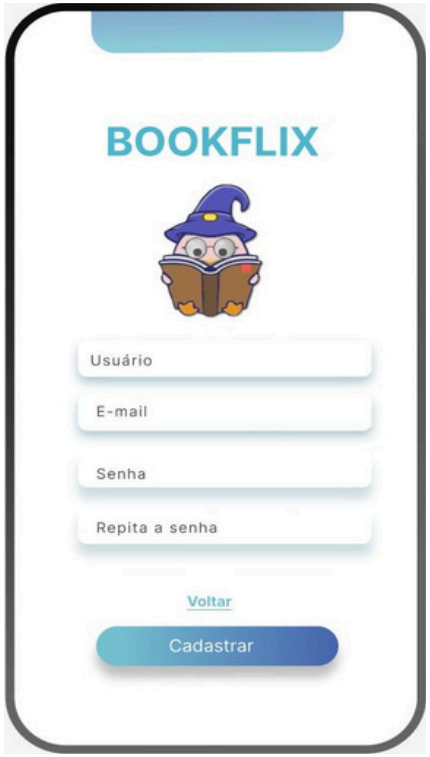
User main page

The app

Screen Prototypes

Essential Screens

Login page



Registration page

The app

Screen Prototypes

Essential Screens

Sidebar menu

Password reset page

Notification examples

Architecture

Database

PostgreeSQL
Relational

Front End

Flutter - Dart
Component based

Back End

Node JS
Express

API

OpenLibrary
RESTful

Architecture

Database

PostgreSQL is an object-relational database system compliant with SQL standards.

Free

Safe

Extensible

```
TABLE user {
 id integer not null nextval
 name varchar(120) not null
 email varchar(120) not null
 password varchar(120) not null
 level integer default 0
 points integer default 0
 current_book varchar(13)
 created_at timestamp current
 genres text[]
}
```

Architecture

BackEnd

Node.js is an asynchronous JavaScript runtime.

Free

High performance

Microservices-friendly

```
app.post('/user', async (req, res) => {
 const { name, email, password,
level, points, current_book,
created_at } = req.body;
 try {
 const userExists = await pool.query('SELECT * FROM user
WHERE email = $1', [email]);
 if (userExists.rows.length > 0) {
 return res.status(400).json({ error: 'Email already in use' });
 }
 const hashedPassword = await hashPassword(password);
 const result = await pool.query(
 'INSERT INTO user (name, email, password, level, points,
current_book, created_at) VALUES ($1, $2, $3, $4, $5, $6,
$7) RETURNING *',
 [name, email, hashedPassword, level, points, current_book,
created_at]
 );
 res.json(result.rows[0]);
 } catch (err) {
 console.error('Error creating user:', err);
 res.status(500).json({ error: 'Internal server error' });
 }
});
```

Architecture

FrontEnd

Flutter is a cross-platform UI framework.

Customisable

Comprehensive

Open Source

```
@override
Widget build(BuildContext context) {

 final double width = MediaQuery.of(context).size.width * 0.9;
 final double height = width * 0.4;

 return Center(
 child: Container(
 width: width,
 height: height,
 padding: const EdgeInsets.all(16),
 decoration: BoxDecoration(
 color: Colors.white,
 borderRadius: BorderRadius.circular(10),
 ),
 child: Row(
 mainAxisAlignment: MainAxisAlignment.spaceEvenly,
 children: [
 buildSummaryItem(Icons.emoji_events, level, 'Level'),
 buildSummaryItem(Icons.star, points, 'Points'),
 buildSummaryItem(Icons.book, books, 'Books Read'),
], ), ), ); }
```

Architecture

API

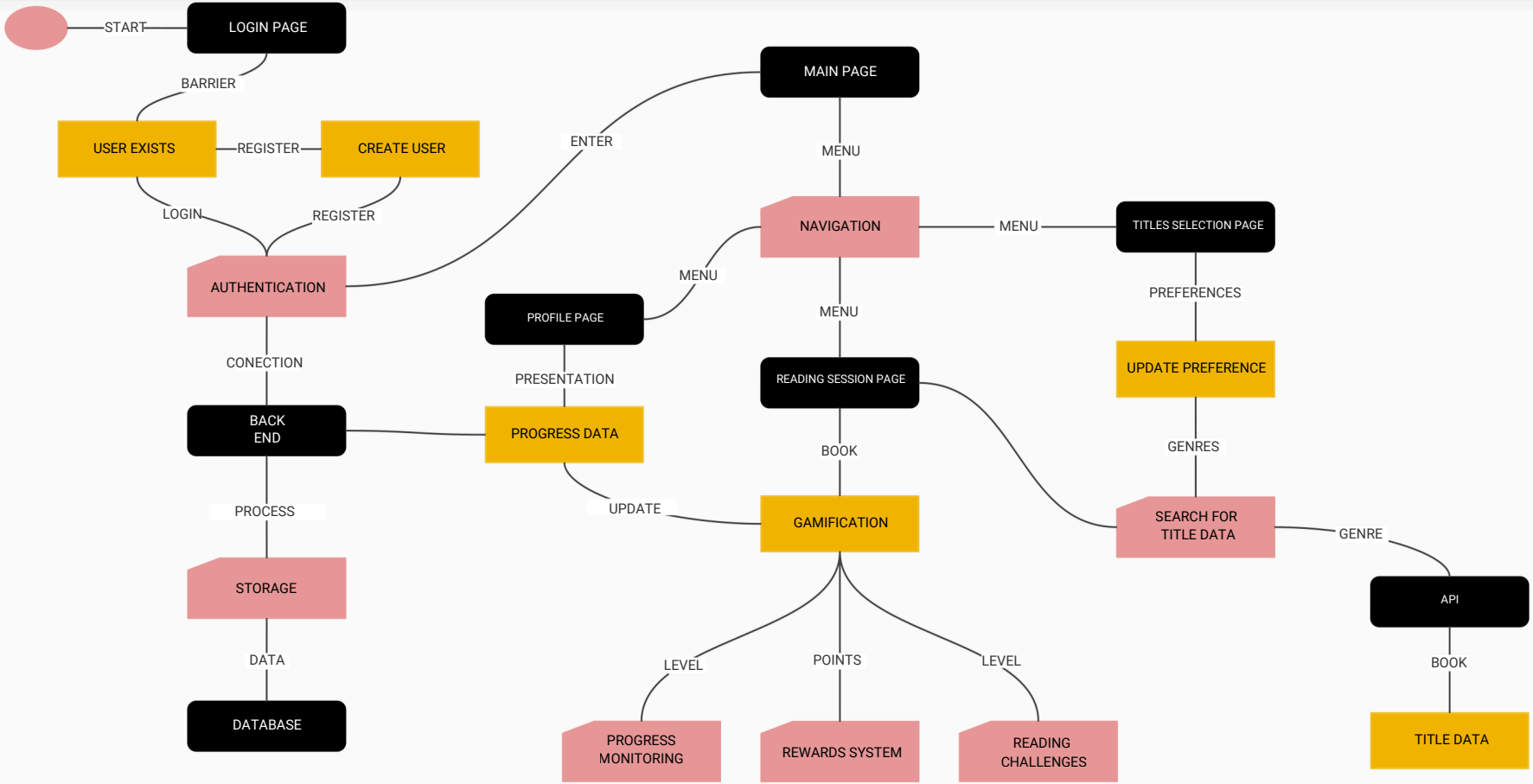
Open Library is an initiative aiming to catalogue all books available on the internet, providing RESTful APIs to access book data.

```
@Future<Book> searchByGenre(String genre,
    List<String> booksShow) async {
        bool found = false;
        while (!found) {
        final response = await http.get(Uri.parse(
'https://openlibrary.org/search.json?subject=$genero&sort=rat
ing&limit=7&fields=title,isbn,cover_i,author_name,publish_date'
 ));

if (response.statusCode == 200) {
final booksDataArray = json.decode(response.body)['docs'];

(...)
```

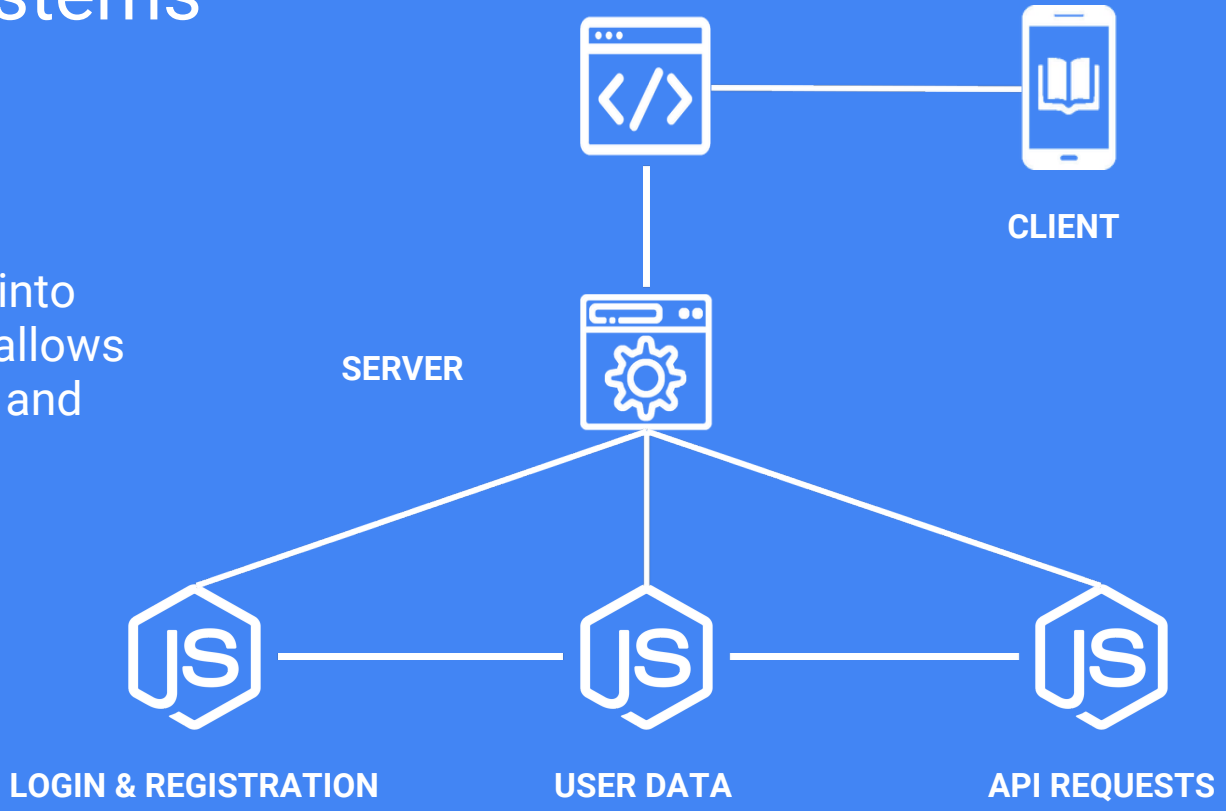
Application Flow



Distributed Systems

Microservices

Splitting processes into independent services allows for better scalability and maintainability.

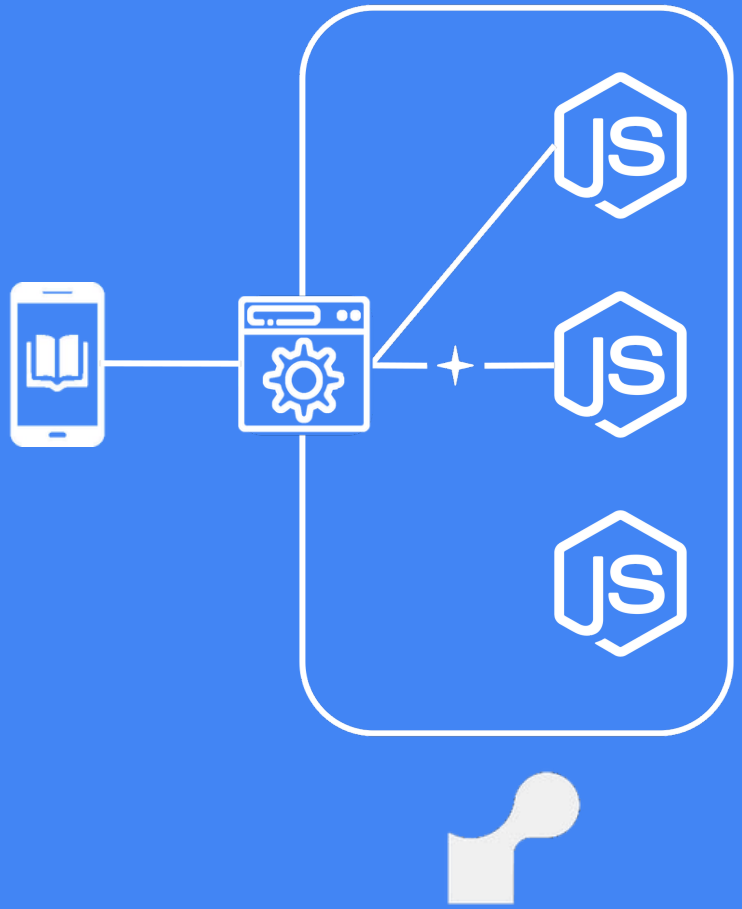


Distributed Systems

Load Balancing

Currently, the prototype is hosted on the Render platform.

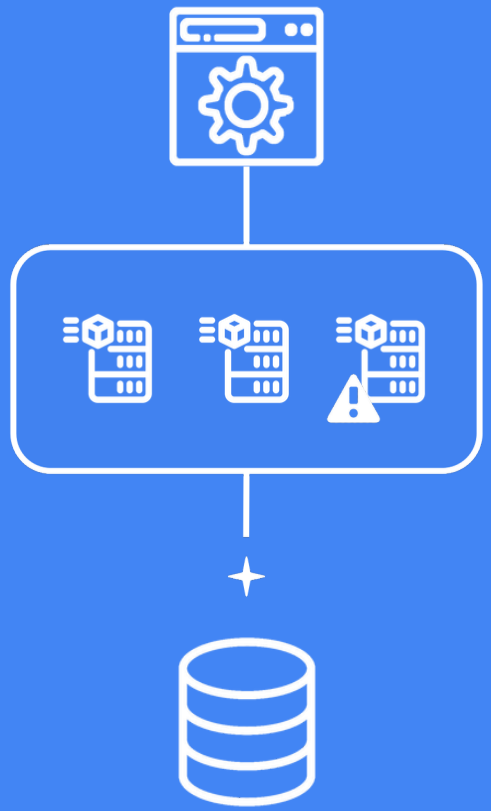
It provides built-in load balancing across instances, enabling initial horizontal scalability.



Distributed Systems

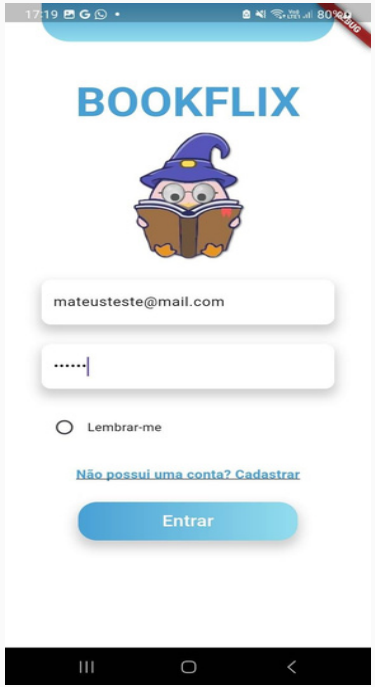
Transactions

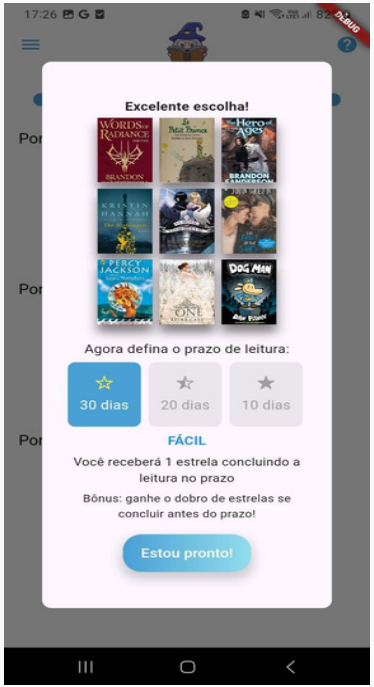
Atomic transactions will be used in the backend for database operations to ensure data integrity and prevent inconsistencies.



Prototype

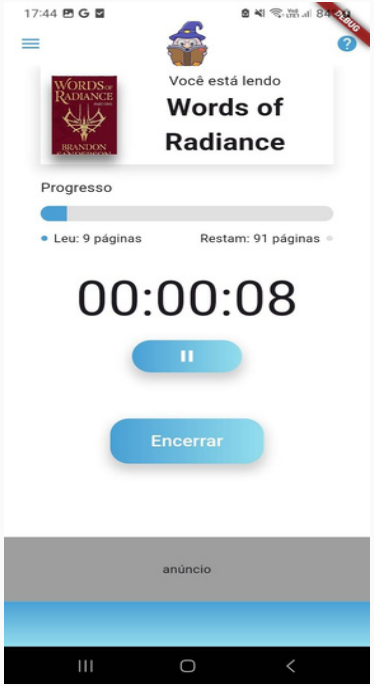
Screens rendered within the application

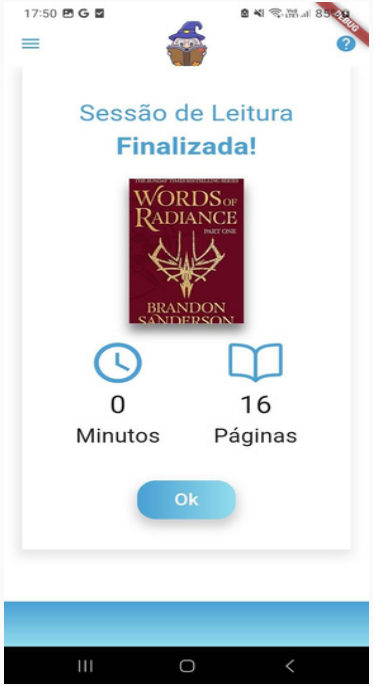




Prototype

Screens rendered within the application

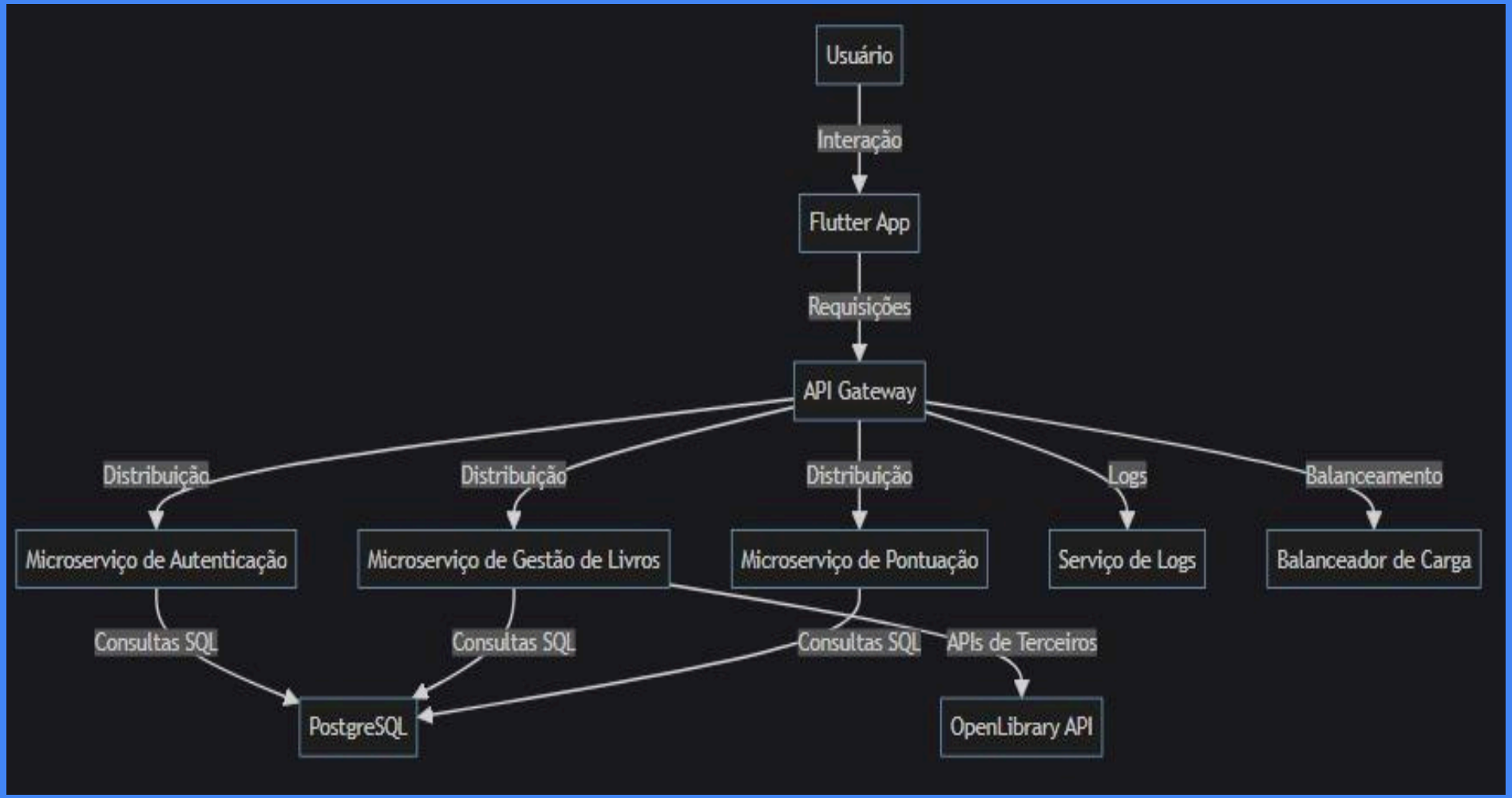




BookFlix

Thank you!

Enjoy your reading!


Usuário
Interação
Flutter App
Requisições
API Gateway
Distribuição
Distribuição
Distribuição
Logs
Balanceamento
Microserviço de Autenticação
Microserviço de Gestão de Livros
Microserviço de Pontuação
Serviço de Logs
Balanceador de Carga
Consultas SQL
Consultas SQL
Consultas SQL
APIs de Terceiros
PostgreSQL
OpenLibrary API